

# OperisChain — Agent Context

## What This Project Is

OperisChain automates document-heavy operational workflows for freight forwarders and logistics brokers in Colombia and LatAm. The first product is a **Quote Generator** that turns messy carrier rate emails into structured, trustworthy quotes.

**Business model:** Implementation consulting (\$12k–\$22k) sold through a \$3k audit. This is a real product with real paying customers — not a side project.

---

## Architecture Philosophy (Read This First)

**The central rule:** LLM as reasoning layer. SQL as truth layer. RAG only for unstructured context.

Freight rate data is structured data trapped in unstructured containers. Treating it like semantic documents (pure RAG) looks impressive in demos and breaks under real data. The architecture enforces this separation strictly.

## The 4 Stages of the Quote Generator

### Stage 1: Deterministic Extraction

Email/PDF/Excel → parse → normalize → INSERT into rates table (PostgreSQL)

LLM may assist extraction but with narrow prompt + confidence\_score

Low confidence = flag for human review, NEVER silent

### Stage 2: SQL Deterministic Filter

User query → SQL WHERE origin/destination/validity/weight → filtered Rate rows

NO vector search for pricing. NO embedding similarity for route matching.

If not in SQL table → tell user explicitly, don't hallucinate

### Stage 3: LLM Reasoning Layer

Filtered Rate rows → LLM ranks, explains, recommends

LLM receives clean pre-validated data — it does NOT extract raw numbers

Output validated with Zod QuoteSchema

### Stage 4: RAG Context Layer

Vector search over full email bodies + contracts → augments Stage 3

Provides: special conditions, exceptions, carrier notes

Does NOT replace SQL results

---

## Stack (Non-Negotiable for v1)

Runtime: Node.js 20 LTS + TypeScript (strict)  
Framework: Next.js 14 App Router (single repo, no monorepo)  
Database: Supabase — PostgreSQL + pgvector + Storage (all in one)  
ORM: Drizzle ORM (drizzle-orm + drizzle-kit)  
LLM 1: OpenAI GPT-4o (extraction + quote generation)  
LLM 2: Anthropic Claude claude-sonnet-4-6 (complex document analysis)  
Chains: LangChain.js LCEL (langchain@0.2)  
Validation: Zod (all LLM outputs MUST be validated with Zod)  
Tracing: LangSmith (every LLM call must be traced — non-negotiable)  
Auth: NextAuth.js with JWT  
UI: shadcn/ui + Tailwind CSS  
Tables: TanStack Table v8  
Streaming: Vercel AI SDK (ai@3)  
Deploy: Vercel

**DO NOT add:** Pinecone (use pgvector), Redis (not needed v1), Turborepo, Hono, Clerk, Cloudflare R2.

**Reason:** Complexity before it's needed. Add only when a specific client requirement demands it.

---

## Non-Negotiable Rules

1. **TypeScript strict mode always.** No `any`. No `as unknown as X` hacks.
  2. **Every LLM output validated with Zod before use.** If it doesn't match schema, throw and log — never pass bad data downstream.
  3. **Every LLM call traced with LangSmith.** No exceptions. Tag with `{ clientId, module, version }`.
  4. **confidence\_score on every LLM extraction.** 0.0–1.0. Below 0.7 = flag for human review.
  5. **surcharge\_flags explicit.** If BAF/FSC/PSS is excluded or unknown, show warning to user. Never hide it.
  6. **Source citation always.** Every rate line in a quote must include `sourceDocId` pointing to the original document.
  7. **SQL for numbers, RAG for context.** Never use vector similarity to find a price. Use SQL.
  8. **Null is honest.** If `valid_until` can't be parsed → null + warning to user. Never guess a date.
- 

## Database Schema (Drizzle)

See `drizzle/schema.ts` — already defined. Key tables:

- `clients` — one row per freight forwarder company
- `documents` — raw ingested files (PDF, Excel, email)
- `rates` — structured extracted rates (the truth layer)
- `quotes` — generated quotes with full audit trail
- `quote_lines` — individual lines within a quote

**Critical fields on `rates`:**

- `confidence_score REAL` — extraction confidence 0.0–1.0
- `surcharges JSONB` — array of `{type, status, note}`
- `valid_until DATE` — nullable, with `validity_warning TEXT`
- `source_doc_id UUID` — always points to original document

**Project Structure**



```

| | └─ queries.ts
| └─ scoring.ts      ← scoreRate(rate, context): number
| └─ surcharges.ts   ← getSurchargeWarnings(surcharges[]): Warning[]
└─ components/
    └─ quote-table.tsx
    └─ surcharge-banner.tsx
    └─ confidence-badge.tsx
    └─ upload-zone.tsx
└─ drizzle/
    └─ migrations/

```

---

## Build Order (Follow This Sequence)

**Week 1:** `lib/extraction/` — parsers + normalizer + rate-extractor + Drizzle schema migrations **Week 2:**

`lib/db/queries.ts` + `lib/scoring.ts` + `lib/surcharges.ts` + unit tests **Week 3:** `lib/chains/extract.chain.ts` +

`lib/chains/quote.chain.ts` + LangSmith tracing **Week 4:** All `components/` + `app/(dashboard)/quotes/` + PDF export

**Week 5:** Real client data test + friction fixes **Week 6:** Polish + deploy + demo prep

**Do not skip steps. Do not build Week 3 before Week 1 is tested with real email data.**

---

## The Trust Threshold

The product succeeds when a freight operator sends a quote to their client **without manually verifying the underlying rates**. Every architectural decision should be evaluated against this question: "Does this make the operator trust the output more?"

- Surcharge warnings → yes, show them
  - Confidence badges → yes, show them
  - Source citations → yes, always
  - Hiding uncertainty → never
- 

## Edge Cases Already Designed

See `lib/extraction/normalizer.ts` for implementations of:

- **Vague validity dates:** "until further notice" → null + warning. "end of month" → last day of month.
- **Route normalization:** "Bogotá – Miami", "BOG→MIA", "Colombia to Florida" → all become `BOG` / `MIA`

- **Currency normalization:** COP, EUR → USD using stored conversion rate
  - **Urgency unknown:** `needsClarification: true` + specific question shown to user before rendering table
  - **Surcharge flags:** Any `excluded` or `unknown` → visible banner with exact surcharge names
- 

## Environment Variables

```
env

OPENAI_API_KEY=
ANTHROPIC_API_KEY=
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=operischain-dev
DATABASE_URL=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_KEY=
NEXTAUTH_SECRET=
NEXTAUTH_URL=http://localhost:3000
```

---

## What NOT to Build (v1 Scope)

- ❌ Email triage module (Week 7+)
- ❌ Document Q&A chat (Week 7+)
- ❌ Compliance / HS code assistant (Week 9+)
- ❌ Multi-tenant admin dashboard
- ❌ Billing / Stripe integration
- ❌ Mobile app
- ❌ Webhook system
- ❌ Any feature not directly serving the Quote Generator trust threshold

If you're about to build something not listed in the Week 1–6 plan, stop and ask.